Revenant: where we are, and your tasks

Everything that changed this week, and what you own from now to submission.

Deadline	Fri 4 Dec, 2:00 PM Pacific (Sat 5 Dec, 3:00 AM Pakistan). We submit on **Thu 3 Dec**.
Category	**Frankenstein**: stitch together a chimera of technologies into one app.
Repo	github.com/Abbas-Rahman-c/**revenant** (renamed from summoning-circle; old links redirect)
You own	The device sigil, the voice and memory parts, eyes, error states, testing, the Kiro power and the launch kit

What changed

- **New name: Revenant**, one who returns from death. It replaces Summoning Circle everywhere: site, title screen, README, Kiro docs and the Devpost thumbnail.
- **New category: Frankenstein.** Five browser AI parts, each on a different web technology, sewn into one creature. This fits the brief better than Skeleton Crew.
- **One creature app.** We dropped the two separate apps (Possessed and 3 AM). Their best ideas live on as creature features: it sees you through the camera, and it remembers you.
- **The whole site is now a 3D lab.** A title screen enters the lab with sound. A real anatomical body (skin, 231 bones, 277 muscles, brain, heart) stands between two Tesla coils under translucent, stitched skin, as the 1818 novel describes it. You stitch parts on from specimen jars, throw a knife switch, lightning runs down two electrode cables, current floods through veins instead of blood, it wakes, and the electrodes pop off. Scrolling moves the camera through the room.
- **It runs on weak PCs too.** Four quality tiers follow the real frame rate, and a lighter body model loads on low-end machines.
- **Credits:** anatomy from BodyParts3D (CC BY 4.0). The credit is in the footer and README and must stay.

Important: loading and speech in the lab are still **simulated**. The 3D and the experience are done; the real AI engine behind them is what we build now, and most of your tasks are part of that.

Get your copy up to date

```
git remote set-url origin https://github.com/Abbas-Rahman-c/revenant.git
git pull
npm install
npm run dev          # then open http://localhost:5173
```

Use Chrome or Edge (they have WebGPU). Click **Enter the lab**; headphones help.

The rule that decides whether we qualify

Judges score **use of Kiro** at 20%, and the rules say too little Kiro use can disqualify us. Right now `.kiro/specs/` is **empty**: the site was built outside Kiro. So from today:

- **Every task below starts as a Kiro spec** (Spec mode): requirements in EARS format, then design, then tasks. Commit the spec in `.kiro/specs/`.
- **Add at least one agent hook per task** (for example: run the tests when a file in your folder is saved).
- **Use property-based tests** where the doc says so. Kiro supports them and judges look for them.
- **Keep notes** in `planning/kiro-notes.md`: what the spec got right or wrong, which hook saved time, what property tests caught. This becomes our write-up.

Your tasks

#	Task	When	Done when
1	**Device sigil.** `circle/src/sigil/`: read the device, choose tiny / small / full. Full spec in `planning/tasks/Task_01_Device_Sigil.pdf`.	Nov 9 to 12	Unit and property tests pass; spec and hook committed; PR into main
2	**Verify the voice and memory models.** Run a real text-to-speech model (candidate: Kokoro 82M) and an embeddings model (candidate: all-MiniLM-L6-v2) in the browser. Record exact versions, model IDs, sizes and licenses in `.kiro/steering/local-ai.md`.	Nov 12 to 15	Two verified entries with working snippets; nothing guessed
3	**Voice and memory parts.** `circle/src/parts/voice.ts` and `memory.ts`: `speak(text)` returns audio; `remember(text)` and `recall(query)` store and search in IndexedDB. Both run in a web worker using the message protocol Abbas writes.	Nov 16 to 22	Both parts work through summon(); tests pass; real download progress reported
4	**Eyes.** `circle/src/parts/eyes.ts` with MediaPipe Object Detector: `detect(video)` returns labels and scores.	Nov 18 to 22	Detects objects from a webcam at 2 or more times a second on a laptop
5	**Problem states, mobile and smoke test.** Screens for: camera or mic denied, no WebGPU, out of storage, download failed, offline first visit. A mobile pass on a real Android phone. A Playwright test: enter, stitch brain and ears, throw, reach 'It's alive'.	Nov 23 to 27	Every state reachable and readable; smoke test passing
6	**Kiro power.** Package our steering docs, hooks and the MCP server Abbas builds into `power/` so anyone can install it.	Nov 23 to 27	Installs into a fresh Kiro and works
7	**Launch kit.** Demo video under 3 minutes (no copyrighted music, no film clips or names), Devpost page, social post starting with **#KiroweenHackathon26**, blog post mentioning Kiroween 2026, Best Student Project details.	Nov 28 to Dec 3	Everything submitted on Dec 3

Before Dec 1, both of us: submit the Kiro Credit Request Form for the free 4,000 credits, if not done yet.

What Abbas is building at the same time

So you know what your parts plug into and when it lands.

When	Abbas
Nov 9 to 15	Verify the brain (language model) and ears (Whisper); write the worker message protocol; `summon()` core
Nov 16 to 22	Brain and ears parts; wire the lab's switch to real downloads
Nov 23 to 27	Talk mode (mic, camera, chat, speech); deploy with the right headers; `circle-mcp` server
Nov 28 to Dec 3	Kiro write-up, README, final fixes

The contract your parts must follow

The site only talks to the engine through this. Build your parts to match it exactly. The full version is in `planning/BACKEND_PLAN.md`.

```
type PartId = "brain" | "eyes" | "ears" | "voice" | "memory";

summon(part: PartId, opts?: {
  tier?: "auto" | "tiny" | "small" | "full";
  onProgress?: (p: { stage: "checking" | "downloading" | "warming-up" | "ready";
                     loadedBytes: number; totalBytes: number; fromCache: boolean }) => void;
  signal?: AbortSignal;
}): Promise<Handle>;            // rejects with a CircleError

voice.speak(text: string): Promise<{ samples: Float32Array; sampleRate: number }>
memory.remember(text: string): Promise<void>
memory.recall(query: string, k?: number): Promise<string[]>
eyes.detect(frame: HTMLVideoElement): Array<{ label: string; score: number }>
handle.banish(): void           // unload and free memory

CircleError.code: "NO_WEBGPU" | "OUT_OF_MEMORY" | "LOW_STORAGE"
                | "DOWNLOAD_FAILED" | "MODEL_LOAD_FAILED" | "ABORTED"
```

Rules for everything we ship

- **No guessed APIs.** Check the official docs, run it, then write the verified entry in `local-ai.md` before writing code.
- **Pin exact versions** of every AI library, and never commit model files (they download at runtime).
- **Honest numbers.** Sizes, speeds and benchmark results are measured, never estimated.
- **Our own creature.** The film photos were mood only: no studio creature designs, no film names, clips or music anywhere, including the video.
- **Private by default.** Camera, microphone and conversation never leave the device. Only model files are downloaded.
- **Small pull requests** into main, one per task. Push half-finished work to your branch rather than sitting on it.

Stuck? Message the group with what you tried. A half-finished branch beats a silent day.